

Ôn trên lớp

- introduction 3 câu:
 - phân biệt 4 phương pháp dịch (2 câu).
 - 1 slide gần cuối (các thành phần ngôn ngữ lập trình: linker, loader, processor, assembler... tác dụng của nó là gì? - ví dụ `#include`, `#define`, linker là kết nối các file dịch lại với nhau...)
- lexical analysis: 8 câu.
 - 6 câu là harmony.
 - Có thể hỏi ngược (mô tả rồi chọn biểu thức chính quy thỏa mãn) hoặc hỏi xuôi (cho RegEx, hỏi chuỗi nào thỏa biểu thức chính quy đó)
 - Harmony: đếm số pattern
 - 2 câu còn lại:
 - Không có automata
 - Điểm tối ưu là gì
 - tập trung (A1-1), câu 1 phần 1 (lexer)
- syntax analysis: 8 câu.
 - 6 câu harmony
 - Theo chiều thuận và nghịch (cho văn phạm, hỏi văn phạm thỏa mãn, hoặc ngược lại)
 - Không harmony: 2 câu
 - vai trò bộ phân tích cú pháp
 - các lỗi là lỗi gì? (lỗi ném ra là lỗi của bộ nào)
 - dẫn xuất trái nhất là gì?
 - Văn phạm có nghĩa lệch
 - Chỉ ra biến thể từ quiz.
- ast generation: 10 câu.
 - 10 câu harmony
 - 2 phần, 1 đoạn tương đối quen (6 câu), 1 đoạn tương đối lạ (4 câu). (phần lớn là BNF, những chỗ không quan trọng là ENBF).
 - Harmony không phải lúc nào là điền khuyết code. cần biết và hiểu đề (với ngữ pháp đó, với đoạn mã này, cây ast sinh ra là gì? số terminal node là bao nhiêu? ví dụ đó đúng hay là sai?)
- OOP: 6 câu
 - tập trung phần kế thừa. (inheritance), từ khóa super (lớp cha), hiểu rõ về nó.

- biến static và không static trong Python
- giải thuật MRO (thứ tự phân loại các lớp).
- Phần OOP trong Python là chính
 - slide case study Python (khái niệm class vs object - định nghĩa một class - static methods (instance, static khác nhau chỗ nào), tập trung nặng vào inheritance (python là ngôn ngữ đa thừa kế, sử dụng MRO để giải bài toán kim cương quyết định thứ tự tìm kiếm của 1 lớp), các từ khóa super, isinstance, type sử dụng như thế nào, không ra polymorphism)
- FP: 6 câu
 - Immutable data (constant, không thay đổi được). so sánh mutable và immutable (list, tuple, ...)
 - First-class Function (hàm xem như 1 giá trị, giống như con trỏ hàm)
 - High-order function hỏi trong harmony.
 - Khái niệm closure (hàm nằm trong môi trường khác) (ứng dụng thực tế vào những chỗ nào)
 - Decorator là gì, ứng dụng vào chỗ nào?
- Name: 4 câu
 - Nội dung nằm hầu hết trong quiz (bài name giữa kì giống như quiz).
 - Bài tập (có trong quiz, chép ở dưới):
 - foo1 thấy x dòng 1, y dòng 4, z dòng 5, foo dòng 2, foo1 dòng 4. (nhớ các tên hàm).
 - Xác định đối tượng gắn với nó: y07, z08, xo1
 - Cấu trúc khối → tầm vực tĩnh (tĩnh = trước khi nó chạy, căn cứ vào văn bản), mới được làm như vậy.
 - Tầm vực động: xác định khi chạy (nhìn luồng main → foo(1) → foo(2) → foo1. Tầm vực động: không quan tâm tới đoạn mã, quan tâm tới thứ nó chạy (trong hình)
 - Nếu là tầm vực tĩnh: căn cứ vào đoạn code.
 - Có 2 dạng bài, 1 là kể tên, 2 là kể đối tượng. Tập trung vào tầm vực tĩnh (là 1 câu trọng kì trước, sửa đúng 1 chữ)

Functional Programming

Bài 1:

Cho biết Python dùng phép toán `**` để tính lũy thừa, ví dụ `2**3` sẽ có kết quả là 8. Hãy điền vào chỗ trống dưới đây để hàm `f` trả về một hàm lũy thừa tương ứng với thông số nhận được? Ví dụ `f(2)` trả về hàm lũy thừa bậc 2, `f(3)` trả về hàm lũy thừa bậc 3. Để đảm bảo so trùng khớp với đáp án, KHÔNG ghi một khoảng trắng nào khi điền ô trống. Lưu ý mỗi chỗ trống không ghi quá 4 ký tự.

```
def f(x):
```

```
    return lambda y:
```

```
square = f(2)
```

```
print(square(3)) # kết quả là 9
```

```
cube = f(3)
```

```
print(cube(2)) # kết quả là 8
```

- Kết quả: `lambda y: y**x` (closure)

Bài 2: viết một hàm tên là hàm `Compose(f, g)` . cho:

$$h = compose(f, g)$$

$$h(x) = f(g(x))$$

giải:

```
def compose(f, g)
    def h(x):
        return f(g(x))
    return h
```

```
f = lambda x: x * 3
g = lambda x: x + 1
```

```
h = compose(f, g)
h(3) # result?
```

- Kết quả:
 - $h(3) = f(g(3)) = f(4) = 4 * 3 = 12$.

Bài 3: viết hàm `compose` tổng quát (hàm hợp của một chuỗi các hàm:

```
def compose(f1, f2, **fx)
```

- `***fx` (bao nhiêu dấu `*` nghĩa là có bấy nhiêu biến. `***fx → f3, f4, f5`) có nghĩa là từ phần tử thứ 3 trở đi sẽ gộp lại thành 1 tuple. phần tử 1 và 2 tạo thành 2 hàm bình thường

Ví dụ: cho `t = compose(f, g, h)`, nếu người ta hỏi `t(x) → f(g(h(x)))`

- Với 3 dấu sao: `Compose(f1, f2, f3, f4, f5) → Compose(f1, f2, fx=(f3, f4, f5))`

(Tham số có 1 dấu `*` trong Python, tham số có 2 dấu `*` trong Python)

Giải:

```
def compose(f1, f2, ***fx)
    f = [f1, f2] + list(fx) # (f(x) là tuple chứa 3 tham số: f3, f4, f5)
    def (out(x)):
        # Chạy ngược: dồn từ f5 → f4 → f3 → f2 → f1 → Đảo thứ tự: f[::-1]
        return reduce(lambda acc, cur: cur(acc), f[::-1], x)
    return out
```

- từng `cur` là từng cái hàm;
- `acc` là tích lũy trước đó - hay gọi là đầu vào cho cái tiếp theo

AST Generation

```
ifstmt: ifclause elifclause elseclause? SEMI;
ifclause: IF expr block; // expr represent an expression
elifclause: ELSE ifclause;
elseclause: ELSE block; // block represents a block
```

cùng với lớp **If** trên AST để biểu diễn cho phát biểu **If** như sau:

```
@dataclass
class If (Stmt):
    cond: Expr
    thenStmt: Stmt
    elseStmt: Stmt #None if there is no else
```

và đoạn mã sau để sinh ra AST cho phát biểu **If** là một đối tượng của lớp trên:

```
1 def visitIfstmt(self, ctx):
2     ifast = self.visit(ctx.ifclause)
3     elifast = list(map(lambda x: self.visit(x), ctx.elifclause))
4     elseast = self.visit(ctx.elseclause)
5     return functools.reduce(lambda x,y: __(1)__, __ (2)__, elseast)
6 def visitIfclause(self, ctx):
7     return (self.visit(ctx.expr), self.visit(ctx.block))
8 def visitElseclause(self, ctx):
9     return self.visit(ctx.block)
```

Cho biết `visitExpr` trả về `Expr` và `visitBlock` trả về `Stmt`.

Câu 10. [A2] Có mỗi lỗi chung xảy ra trên tất cả phương thức *visit* đã cho. Lỗi đó là:

- ☐ (A) thiếu khai báo kiểu cho ctx, tức phải viết `ctx:MiniGoParser.IfclauseContext`.
- ☐ (B) nhầm lẫn khi sử dụng `self` và `ctx`, tức phải viết `ctx.visit(self.ifclause)` thay vì `self.visit(ctx.ifclause)`
- ☐ (C) phải dùng `accept` thay vì `visit`, tức phải viết `ctx.ifclause.accept(self)` thay vì `self.visit(ctx.ifclause)`
- ☐ (D) phải gọi phương thức của `ctx`, tức phải viết `self.visit(ctx.ifclause())` thay vì `self.visit(ctx.ifclause)`

Câu 11. [A2] Sau khi đã sửa lỗi chung đề cập ở câu trên, lỗi *AttributeError: "NoneType" object has no attribute 'accept'* xảy ra khi sinh mã AST cho một phát biểu **If** đúng. Hãy cho biết lỗi này xảy ra ở dòng nào (chọn dòng đầu tiên có thể gây ra lỗi này)

- ☐ (A) 4
- ☐ (B) 2
- ☐ (C) 3
- ☐ (D) 7

Câu 12. [A2] Hãy chọn mã thích hợp cho chỗ trống (2):

- ☐ (A) `elifast + [ifast]`
- ☐ (B) `elifast[::-1] + [ifast]`
- ☐ (C) `[ifast] + elifast`
- ☐ (D) `[ifast] + elifast[::-1]`

Câu 13. [A2] Hãy chọn mã thích hợp cho chỗ trống (1):

- ☐ (A) `If(y[0],x,y[1])`
- ☐ (B) `If(y[0],y[1], x)`
- ☐ (C) `If(x[0],y,x[1])`
- ☐ (D) `If(x[0],x[1],y)`

→ If Statement.

10D (ctx. tên → gọi một method phải ngoặc)

11. NonType: giá trị hiện tại đang gọi tới đang mang giá trị None. thằng nào trả về None thì chọn →
Đáp án A (dòng số 4) (có thể kiểm tra có hay không có)

12. B

13. B

Cho văn phạm viết trên ANTLR4 mô tả các khai báo thông số trong một khai báo hàm của MiniGo như sau:

```
params: param (CM param)*;  
param: ids mtype; // mtype represents a type  
ids: ID CM ids | ID;
```

cùng với lớp ParamDecl trên AST để biểu diễn

```
@dataclass  
class ParamDecl (Decl):  
    parName: str # name of one parameter  
    parType: Type
```

và đoạn mã sau để sinh ra AST cho các khai báo thông số như ruột danh sách các đối tượng ParamDecl:

```
def visitParams(self, ctx: MiniGoParser.ParamContext):  
    return __ (1) __  
def visitParam(self, ctx: MiniGoParser.ParamContext):  
    a = ctx.getChild(0).accept(self)  
    b = ctx.getChild(1).accept(self)  
    return __ (2) __  
def visitIds (self, ctx: MiniGoParser.IdsContext):  
    id = ctx.ID().getText()  
    if __ (3) __ == 3: return [id] + __ (4) __  
    return __ (5) __  
def visitMtype(self, ctx: MiniGoParser.MtypeContext): # return Type
```

Câu 14. [A2] Hãy chọn mã thích hợp cho chỗ trống (4):

- ☐ A ctx.ids.accept(self) ☐ B self.visit(ctx.ids()) ☐ C ctx.visit(self.ids()) ☐ D self.visit(ctx.ids)

Câu 15. [A2] Hãy chọn mã thích hợp cho chỗ trống (5):

- ☐ A id ☐ B self.visit(ctx.ID()) ☐ C ctx.visit(self.ID()) ☐ D [id]

Câu 16. [A2] Hãy chọn mã thích hợp cho chỗ trống (3):

- ☐ A self.perChildCount() ☐ B ctx.getChildCount() ☐ C ctx.getChild() ☐ D ctx.getChildren()

Câu 17. [A2] Hãy chọn mã thích hợp cho chỗ trống (2):

- ☐ A ParamDecl(i,b) for i in a ☐ B ParamDecl(b, i) for i in a
☐ C ParamDecl(a,i) for i in b ☐ D ParamDecl(i, a) for i in b

Câu 18. [A2] Hãy chọn mã thích hợp cho chỗ trống (1):

- ☐ A functools.reduce(lambda x,y: x+y.accept(self),ctx.param(),[])
☐ B functools.reduce(lambda x,y:x.accept(self)+y.ctx.param(),[])
☐ C list(map(lambda x:self.visit(x),ctx.param()))

→

super

Câu 41. [L.O.3.1] Kết quả của E().show() dựa vào đoạn mã Python sau là gì?

```
class A:  
    def show (self): print("A", end=" ")  
class B (A):  
    def show(self):  
        super().show()  
        print("B", end=" ")  
class C (A):  
    def show (self):  
        print("C", end=" ")  
        super().show()  
class E (B, C):  
    def show (self):  
        print("E", end=" ")  
        super().show()
```

- ☐ A EABCA ☐ B EBCA ☐ C EAB ☐ D ECAB

- Đáp án 242

Mã đề thi 2420		ĐÁP ÁN		
Câu 1. A	Câu 10. D	Câu 19. C	Câu 28. B	Câu 39. D
Câu 2. C	Câu 11. A	Câu 20. B	Câu 30. C	Câu 40. D
Câu 3. B	Câu 12. B	Câu 21. B	Câu 31. B	Câu 41. D
Câu 4. C	Câu 13. B	Câu 22. A ₁	Câu 32. C	Câu 42. C
Câu 5. B	Câu 14. B	Câu 23. D	Câu 33. D	Câu 43. D
Câu 6. B	Câu 15. D	Câu 24. B	Câu 34. D	Câu 44. C
Câu 7. A	Câu 16. B	Câu 25. C	Câu 35. C	Câu 45. D
Câu 8. A	Câu 17. A	Câu 26. A	Câu 36. A	
Câu 9. A	Câu 18. A	Câu 27. C	Câu 37. C	
		Câu 28. A	Câu 38. D	

- Đối tượng mutable (có thể thay đổi): list, set,
- Đối tượng Immutable (không thể thay đổi): **Tuple, int, float, str...**